07 · Tracing und Timing-Analyse

Beobachten, ohne die Messung mit der Ursache zu verwechseln · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie 1 / 42

Willkommen in der Systemanalyse!

- Siebter Termin: eine Messfrage stellen, eine Quelle wählen, die Grenze nennen
- Zielchip wie in VL6: STM32H755, M7 Bare-Metal, M4 FreeRTOS
- Ein Trace ist Evidenz für **beobachtete** Läufe
- Er ist kein WCET-Beweis und kein Normenstempel
- Im Repo liegen kein winIDEA-Projekt und kein BlueBox-Modell

Notizen 1 / 42

Die Leitfrage lautet: Was geschieht wann, und was belegt die Beobachtung wirklich? Das Ping-Pong-System aus Vorlesung 6 bleibt das Beispiel. Der M7 läuft Bare Metal, also ohne RTOS-Kernel. Der M4 nutzt FreeRTOS über CMSIS-RTOS2. RTOS heißt Real-Time Operating System. Ein Trace ist eine geordnete Aufzeichnung von Ereignissen. Vollständigkeit und Zeitbasis müssen geprüft werden. WCET, Worst-Case Execution Time, ist eine obere Schranke unter benannten Annahmen. Ein Trace einzelner Läufe ist diese Schranke nicht.
Ein Heisenbug ist ein Fehler, dessen Erscheinung sich durch die Beobachtung ändert, weil Code, Layout oder Zeit sich ändern. Das ist keine Quantenbehauptung. Drei Lernziele: ein Verfahren wählen, Zeitgrößen auseinanderrechnen und die Grenze der Aussage nennen. Eine fehlende BlueBox ist eine offene Laborentscheidung, kein Hindernis für die Begriffe.
Die Agenda führt vom Einfluss der Beobachtung zur Auswertung.

Folie 2 / 42

Agenda für heute

1. Einfluss der Beobachtung
2. CoreSight: ITM, DWT, ETM
3. Was eine Zahl bedeutet
4. Werkzeugkette und Zeitbasis
5. OS-Sicht und Labor, mit offenen Hardwarepunkten

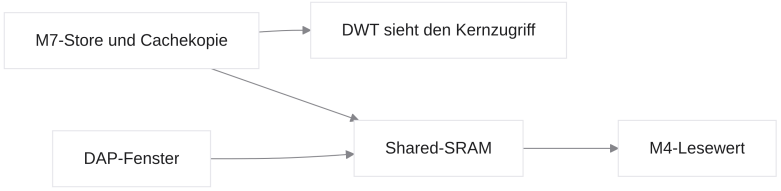
Notizen 2 / 42

ITM, Instrumentation Trace Macrocell, nimmt Softwaredaten entgegen. DWT, Data Watchpoint and Trace, meldet konfigurierte Kernereignisse. ETM, Embedded Trace Macrocell, trägt komprimierten Kontrollfluss. OS ist das Betriebssystem, hier der M4-Kernel. Die Reihenfolge ist absichtlich: zuerst, wie Beobachtung den Ablauf verändert, dann wo Ereignisse entstehen, dann was Zeitmarken bedeuten, zuletzt wie Werkzeuge sie darstellen.
Leitfrage für den Tag: Eine M4-Task reagiert spät. Ist das Rechenzeit, Wartezeit, Scheduling oder ein Messartefakt? Die UART-Rechnung mit „Hallo“ und etwa 434 µs gehört zur printf-Folie. Sie ist Leitungszeit, nicht CPU-Zeit, und ersetzt diese Agenda nicht.
Die Ausgangsarchitektur liefert mehrere Beobachtungspunkte.

Folie 3 / 42

Recap: Unser Multicore-System

- VL6: Ownership, Barriere, non-cacheable Region
- Heute beobachten wir denselben Vertrag
- DWT sieht konfigurierte **CPU-Zugriffe** des gewählten Kerns
- DWT beweist nicht, was im SRAM hinter dem M7-Cache steht
- Ein Dual-Core-Debug beweist weder parallelen Trace noch SWO



Notizen 3 / 42

Drei Orte bleiben getrennt. Der M7 kann eine Cachekopie haben. SRAM, Static Random Access Memory, ist der gemeinsam erreichbare Speicher. Der M4 liest seine eigene Sicht. DWT beobachtet konfigurierte Zugriffe des gewählten Kerns. MPU, Memory Protection Unit, macht im Basislabor die Shared-Region nicht cachebar. IPC, Inter-Processor Communication, ist der vereinbarte Kanal aus Vorlesung 6: Besitz, Barriere, dann der Zustand. Ein Paket „M7 schreibt 1“ belegt diesen Kernzugriff, nicht die aktuelle Lesekopie des

Notizen, Fortsetzung 3 / 42

M4 und nicht jeden Speicherweg. Ein DAP-Fenster, Debug Access Port, ist eine weitere Sicht und kann den Bus belasten. Dass beide Kerne debuggbar sind, heißt nicht, dass zwei Tracequellen gleichzeitig aufgenommen und korreliert decodiert werden.
Jede gewählte Beobachtung hat eigene Eingriffe und Grenzen.

Folie 4 / 42

Beobachtung verändert Softwareabläufe

Analogie, kein physikalisches Gesetz für jede Messung:

- extra Befehle, Logs, anderes Scheduling
- anderes Layout, anderer Cachezustand
- Halt, Buszugriffe des Debuggers, voller Tracepuffer

GPIO, Zykluszähler, ITM, DWT und ETM beantworten verschiedene Fragen und stören verschieden stark.

Notizen 4 / 42

Die Analogie ist kurz: Beobachten kann den Gegenstand verändern. In Software sind das zusätzliche Befehle, Logpuffer, Locks, Buszugriffe, ein anderes Layout oder Cachebild und ein angehaltener Kern. GPIO, General-Purpose Input/Output, setzt eine Pinflanke. ITM nimmt Softwarestimuli entgegen. DWT und ETM können Ereignisse ohne diese Marker liefern, brauchen aber Aktivierung, Filter und einen Puffer. „Ohne zusätzliche Anwendungsbefehle und ohne Halt“ ist eine mögliche Eigenschaft eines Hardwarepfads, nicht der Satz „Einfluss exakt null“. Ein voller Puffer kann verlieren oder, je nach Aufbau, den Kern verzögern. ITM-Softwarestimuli sind instrumentiert. Der ganze ITM-Baustein ist deshalb nicht pauschal „immer nur Software“, weil weitere Ereignisquellen über ihn laufen können.
Zwei Mechanismen: Eine Ausgabe verlängert eine Taskphase. Ein Debughalt ändert die Reihenfolge zwischen den Kernen.
printf liefert dafür ein Größenbeispiel.

Folie 5 / 42

Klassisches Debugging: Die `printf`-Gefahr

„Hallo“, 8N1: 5 Byte = 50 Bit.

$$50/115200 \approx 434\,\mu s$$

Das ist die **Übertragungsdauer**, nicht automatisch CPU-Zeit und nicht der Halt des Chips.

Weg	CPU während der Bits
Polling	bleibt in der Ausgabe
IRQ	kurz, UART läuft weiter
DMA	Start kurz, Transfer ohne CPU

Formatierung, Locks und Puffer kommen dazu. `printf` ist auf dem zeitkritischen Messpfad ungeeignet. Außerhalb bleibt begrenztes Logging möglich.
Ein verpasster Termin ist ein Deadline Miss, kein automatischer Crash.

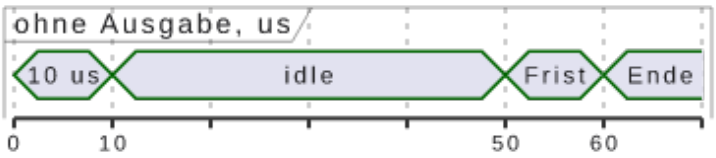
Notizen 5 / 42

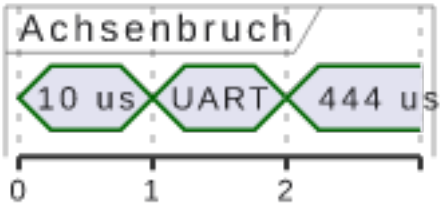
UART, Universal Asynchronous Receiver/Transmitter, ist die serielle Leitung. 8N1 heißt acht Datenbits, keine Parität, ein Stopbit. Mit dem Startbit sind das zehn Bits je Byte. „Hallo“ sind fünf ASCII-Bytes, ohne Nullterminator und ohne Zeilenumbruch. 5 mal 10 gleich 50 Bit. 50 geteilt durch 115200 Bit je Sekunde sind etwa 434 µs. Das ist die reine Leitungsdauer, nicht die CPU-Zeit der Formatierung und nicht ein Halt des Chips. IRQ ist die Interruptanforderung, DMA der direkte Speicherzugriff. Polling belegt die CPU während der Bits. IRQ und DMA verkürzen die CPU-Phase, sind aber nicht aufwandfrei. Treiber und Locks können trotzdem blockieren.
Im Lehrbild: 10 µs Rechnen plus 434 µs blockierende Ausgabe ergeben etwa 444 µs gegen eine angenommene Frist von 50 µs. Ein verpasster Termin ist ein Deadline Miss, kein automatischer Absturz.
Ein Zeitdiagramm macht die Verlängerung sichtbar.

Folie 6 / 42

Visualisierung: Der Beobachter-Effekt

Synthetisches Modell, keine Messung. Die zweite Spur bricht die Achse: gleiche Kästchen sind nicht gleiche Zeiten.





Variante	Inhalt	Dauer	Frist 50 µs
ohne Ausgabe	Rechnung	10 µs	gehalten
mit blockierendem „Hallo“	10 µs plus 434 µs Leitung	etwa 444 µs	gerissen

Der GPIO-Puls von HIGH bis LOW ist die Zeit zwischen physischen Pinflanken. Präemption, Bus, Pad und die Auflösung des Messgeräts liegen darin. Das Messgerät sieht nicht die beiden CPU-Storezeitpunkte.

Notizen 6 / 42

Die Tabelle ist ein synthetisches Modell, keine Messung. Ohne Ausgabe dauert die Rechnung 10 µs und hält die Frist von 50 µs. Mit blockierendem „Hallo“ kommen etwa 434 µs Leitung dazu, zusammen etwa 444 µs. Der lange Balken ist nicht maßstabsgleich zum kurzen. Ein Achsenbruch ist deshalb benannt. GPIO ist der Pin, BSRR, Bit Set/Reset Register, ein direkter Set- oder Reset-Zugriff, HAL die Abstraktionsschicht. Der gemessene Puls läuft von der physischen steigenden zur fallenden Flanke. Präemption zwischen den Marken liegt darin. Bus, Pad und die Auflösung des Messgeräts auch. Das Gerät sieht nicht die beiden Store-Zeitpunkte der CPU. Eine reine Store-zu-Store-Zeit wäre eine andere Definition.
Frage: Warum kann der GPIO-Puls länger sein als die CPU-Zeit der Funktion? Weil Unterbrechungen zwischen den Marken liegen und die Flanke später am Pin ankommt als der Store.
Ein Breakpoint greift noch stärker ein.

Folie 7 / 42

Klassisches Debugging: Breakpoints

- Der **betroffene** Kern hält an
- Der andere Kern, DMA und Peripherie können weiterlaufen
- DBGMCU kann einzelne Timer oder Watchdogs einfrieren, wenn das so konfiguriert ist
- Der NVIC ist keine unbegrenzte FIFO: Pending kann Flanken zusammenfassen
- Puffer können überlaufen
- Pending und Peripherie-FIFOs speichern nur begrenzt. Viele Flanken erzeugen nicht automatisch viele verschachtelte Stackframes.

Nach Resume ist nur rekonstruierbar, was noch in Registern, FIFOs oder Pending steht.

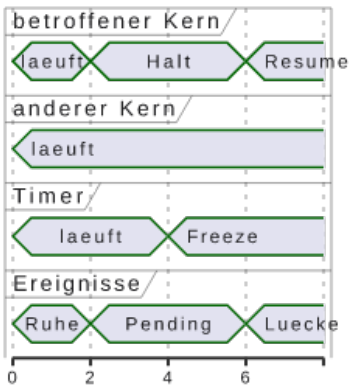
Notizen 7 / 42

Ein Breakpoint hält den betroffenen Kern an. Der andere Kern, DMA und die Peripherie können weiterlaufen. DBGMCU ist der Debug-Support-Block des STM32 und kann einzelne Timer oder Watchdogs einfrieren, wenn das Bit gesetzt ist. Der NVIC, Nested Vectored Interrupt Controller, ist keine unbegrenzte FIFO. FIFO heißt First In, First Out. Ein Pending-Bit merkt sich, dass etwas anliegt, nicht jede Flanke als eigene Zahl. Peripherie-FIFOs und Statusregister haben eigene Grenzen. Viele Flanken erzeugen nicht automatisch viele verschachtelte Stackframes, weil der Kern die ISRs nicht betritt. ISR ist die Behandlungsroutine. Nach Resume ist nur da, was Register, FIFOs oder Pending noch halten. Verlorene Flanken kommen nicht zurück.
Frage: Welche Ereignisse sind nach dem Weiterlaufen noch da? Nur die, die ein begrenzter Speicher noch enthält, nicht jede verlorene Flanke.
Das folgende Bild trennt anhaltende Hardware von angehaltener Software.

Folie 8 / 42

Das Problem des “Halt-States”

Lehrschema. Freeze gilt nur, wenn das passende Bit gesetzt ist. Es gibt keine unbegrenzte Interrupt-Warteschlange.



Notizen 8 / 42

Die Grafik zeigt den haltenden Kern und daneben UART, DMA und den anderen Kern, die weiterlaufen können. Externe Flanken landen im Pending und nicht als unbegrenzte Liste. Eine zweite Variante friert den Timer nur, wenn das Freeze-Bit gesetzt ist. DAP, Debug Access Port, und SWD, Serial Wire Debug, sind der Debugzugriff. Ein laufendes Speicherlesen über DAP ist kein Breakpoint. SWD heißt nicht, dass jeder Zugriff den Kern anhält. Der Verkehr kann trotzdem den Bus belasten und ist keine lückenlose Vergangenheit. Resume stellt verworfene Daten nicht wieder her. Cross-Trigger, der weitere Kerne mit anhält, ist eine Konfiguration, kein Automatismus. UART ist die serielle Schnittstelle, DMA der Kopierer. Halt, DAP-Lesen, Softwarelog und Hardwaretrace sind vier verschiedene Eingriffe.
Solche Eingriffe können einen Fehler verschwinden lassen.

Folie 9 / 42

Der Heisenbug

- Der Fehler verschwindet, weil die Beobachtung die Zeit oder das Layout ändert
- Race: verschränkte Zugriffe ohne Vertrag
- Priority Inversion: ein Lock plus Verdrängung des Owners
- Beides kann heikel zu beobachten sein, beides ist nicht dasselbe

Notizen 9 / 42

Ein Heisenbug verschwindet, weil die Beobachtung Zeit oder Layout ändert. Zwei Beispiele bleiben getrennt. Race: zwei Teilnehmer schreiben ohne gültigen Vertrag, und ein Log verschiebt ihre Reihenfolge. Priority Inversion: eine hohe Task wartet auf ein Lock, das eine niedrige hält, und eine mittlere verdrängt den Besitzer. Lock ist die Sperre, Owner der Besitzer, Präemption die Verdrängung. RTOS ist der Kernel, der diese Zustände führt. Beides kann schwer zu beobachten sein. Es ist nicht derselbe Mechanismus. Dass der Fehler im Debugger weg ist, beweist die Ursache nicht.
Kleine Softwaremarkierungen können gezielter sein. Auch sie müssen selbst untersucht werden.

Folie 10 / 42

Der Workaround: Software Instrumentierung

- GPIO und Zähler beantworten „wann bin ich hier?“
- ITM schreibt Stimulus-Wörter, es formatiert keinen Text
- `ITM_SendChar` kann auf freie Kapazität **warten**
- Voller Puffer ist keine „winzige Seltenheit“
- Laborvariante, wenn wir sie nutzen: nicht blockieren, Drops zählen, Statusbits der vorhandenen CMSIS-Version prüfen

Notizen 10 / 42

Ein Marker ist zusätzliche Arbeit im Zielprogramm. GPIO setzt eine Flanke. ITM, Instrumentation Trace Macrocell, nimmt Datenwörter an einem Stimulusport entgegen und formatiert nicht selbst den Text. CMSIS ist die Arm-Schnittstelle. `ITM_SendChar` kann in der vorhandenen Implementierung auf freien Platz warten. Eine nicht blockierende Lehrvariante macht höchstens einen Versuch, zählt bei fehlender Kapazität einen Drop und wartet nicht. Mehrere Schreiber oder eine ISR brauchen einen eigenen Vertrag. `volatile` macht den Zähler nicht racefrei. SWO, Serial Wire Output, ist der serielle Traceausgang und muss samt Empfänger nachgewiesen sein, bevor wir ihn zusagen.
Manche Fragen brauchen Ereignisse ohne solche Marker.

Folie 11 / 42

Wir brauchen: Non-Intrusive Observation

Präzise Formulierung:

- ohne extra Anwendungsbefehle
- ohne den Kern anzuhalten
- nicht: Einfluss immer exakt null

Prüfen: Trace an, Pfad, Puffer, Überlauf, Stall.

Notizen 11 / 42

Non-intrusive meint hier: bestimmte Ereignisse ohne zusätzliche Anwendungsbefehle und ohne Halt des Kerns. Aktivierung, Filter, Transport und Speicher bleiben nötig. DWT und ETM können solche Kernereignisse liefern. ITM-Softwarestimuli gehören nicht in diese Definition, weil die Anwendung schreibt. Andere ITM-Funktionen dürfen deshalb nicht pauschal als „immer Software“ abgetan werden. Ein voller Puffer kann verlieren. Ob er den Kern anhält, einen Stall, hängt vom Aufbau ab. Das sagen wir nicht für jeden Recorder.
Die Zusammenfassung ordnet die vier Beobachtungsarten.

Folie 12 / 42

Zusammenfassung Block 1

- 434 µs sind Leitungszeit von „Hallo“
- Halt stoppt nicht den ganzen Chip
- GPIO misst ein Intervall, keine reine CPU-Zeit
- Hardwaretrace ist eine Quelle mit eigenen Verlusten

Notizen 12 / 42

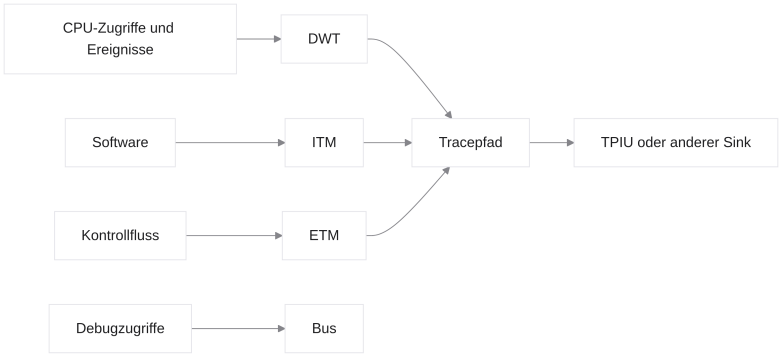
Vier Ergebnisse: Die UART-Leitung von „Hallo“ dauert etwa 434 µs. Ein Halt verändert den Systemzustand und stoppt nicht den ganzen Chip. Ein GPIO-Puls ist ein Intervall zwischen Pinflanken, inklusive Präemption.

Notizen, Fortsetzung 12 / 42

Ein Hardwaretrace ist eine gefilterte Ereignisquelle mit eigenen Verlusten, keine vollständige Historie. UART, GPIO, ITM, DWT und ETM bleiben die Namen dieser Wege.
Frage: Warum kann der GPIO-Puls länger sein als die CPU-Zeit der Funktion? Weil Unterbrechungen zwischen den Marken liegen und Pad sowie Messgerät dazukommen.
Jetzt trennen wir die Tracequellen und ihre Wege.

Folie 13 / 42

Hardware-Trace: ARM CoreSight



Nicht drei gleiche Spione am SRAM. DWT sitzt an der konfigurierten Quelle **des Kerns**.
Was fehlt: DMA-Schreiben, der andere Kern, Cachezeilen hinter dem Store, alles außerhalb des Filters.

Notizen 13 / 42

CoreSight ist die Debug- und Tracearchitektur von Arm. ITM nimmt Softwarestimuli, DWT konfigurierte Kernereignisse, ETM den Kontrollfluss. TPIU, Trace Port Interface Unit, kann der Ausgang der Tracedaten sein. Ein Sink ist der Empfänger oder Speicher, nicht zwingend die TPIU. DAP, der Debug Access Port, liest und schreibt über den Bus und ist kein Ereignisstrom. Die Grafik ist ein Konzept, kein verdrahtetes Nucleo. Ob DWT, ITM und ETM auf dem H755 genau so verbunden sind, prüft RM0399 der Projektrevision. DWT sieht nicht automatisch DMA und nicht den anderen Kern. Gleichzeitiger Trace beider Kerne ist offen.
Debugschnittstelle, Traceausgang und Stecker sind verschiedene Ebenen.

Folie 14 / 42

JTAG vs. SWD vs. Trace

Weg	transportiert	beweist nicht
SWD/JTAG	Debug	lückenlosen Trace
SWO	oft ITM/DWT	volles ETM
parallele Trace-Pins	hohe Rate, wenn vorhanden	dass das Nucleo sie hat
interner Puffer	kurz, on-chip	lange Historie

UM2408: STDC14/MIPI10 am NUCLEO-H755ZI-Q ist kein Nachweis eines 20-poligen parallelen Trace-Steckers. Pinout, Raster und Adapter am Schaltplan prüfen. Pinmux, ST-LINK-Trennung, GND, VTref, Pegel, Reset.

Notizen 14 / 42

JTAG, Joint Test Action Group, und SWD, Serial Wire Debug, sind Debugprotokolle. SWO, Serial Wire Output, ist ein serieller Traceausgang und nicht die SWD-Datenleitung. Bei manchen Cortex-M teilt sich SWO den Pin mit JTAG-TDO. Paralleler Trace braucht einen anderen, bestätigten Pfad. GND ist Masse, VTref die Referenzspannung des Targets, Reset die Rücksetzleitung. Die Tabelle trennt, was transportiert wird, von dem, was nicht bewiesen ist. UM2408 beschreibt am NUCLEO-H755ZI-Q STDC14 und MIPI10. Das ist kein Nachweis eines 20-poligen parallelen Trace-Steckers. Pinout und Adapter stehen im Schaltplan der Boardrevision. Ein interner Puffer ist implementationsabhängig und kurz. Einen Stecker, den das Board nicht hat, zeichnen wir nicht.
ITM zeigt als Nächstes, wie Softwaremarker in den Pfad gelangen.

Folie 15 / 42

Modul 1: Das ITM (Instrumentation Trace)

- Stimulus-Port
- Software muss schreiben
- Warten oder Verwerfen ist eine Implementierungsentscheidung
- Drops dokumentieren, sonst sieht die Lücke wie „nichts passiert“ aus

Notizen 15 / 42

ITM heißt Instrumentation Trace Macrocell. Ein Stimulusport ist ein adressierter Eingang für Softwaredaten. Die Anwendung schreibt, die ITM bildet Pakete, der Transport bringt sie zum Empfänger, die Software stellt sie dar. ITM formatiert nicht selbst „Hallo“. CMSIS-`ITM_SendChar` kann warten. Eine andere

Notizen, Fortsetzung 15 / 42

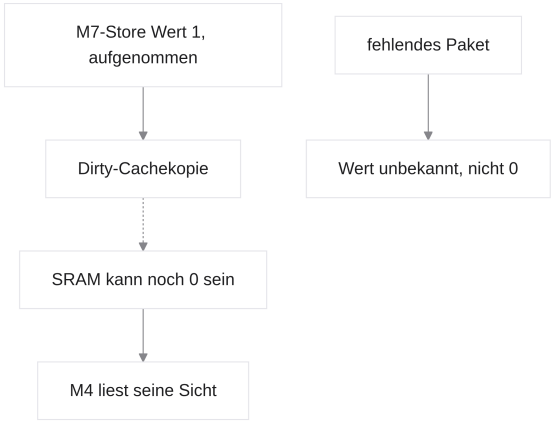
Implementierung kann verwerfen. Ein Dropzähler, ein ausgeschalteter Kanal und ein fehlender Empfänger sind drei Zustände. Der Zähler sagt nicht, welche Ereignisse fehlen. Die Softwarestimuli sind instrumentiert. Der Baustein kann weitere Ereignisse transportieren. Die Bits der Cube-CMSIS-Version benennen wir erst, wenn die Header vorliegen. SWO ist der mögliche serielle Ausgang.
DWT erzeugt dagegen Ereignisse aus konfigurierten Kernzugriffen.

Folie 16 / 42

Modul 2: Das DWT (Data Watchpoint & Trace)

- Wenige Komparatoren, Konflikt mit Watchpoints
- Adresse, Breite, Richtung, Maske, Datenwert und Zeitstempel hängen von der Implementierung ab
- Sie sehen nicht automatisch den anderen Kern oder DMA
- Ein M7-Store „1“ kann im Trace stehen, während Write-Back den Wert dem M4 vorenthält
- Eine flache Kurve beweist nicht „SRAM blieb 0“

Fehlende Pakete können auch heißen: falscher Kern, falsche Adresse, zu spät aufgenommen, Filter, Verlust, Kapazität.



Notizen 16 / 42

DWT heißt Data Watchpoint and Trace. Die Komparatoren sind wenige und können mit Watchpoints kollidieren. Ein vollständiger Filter nennt Kern, Adresse, Breite, Richtung und ob Wert oder Zeit im Paket steckt. Die Adresse kommt aus `offsetof` und der Map, nicht aus einem geratenen Symbol. ELF ist das geladene Image. CPU ist der beobachtete Kern. DMA und der andere Kern sind unsichtbar. SRAM ist der Speicher, den der M4 sehen kann. Ein M7-Store mit Wert 1 kann in einer Dirty-Cachekopie liegen. Die MPU-Basis aus Vorlesung 6 macht den Lehrkanal nicht cachebar. Eine flache Kurve ohne Anfangspaket ist kein Beweis, dass der SRAM 0 blieb. Ein Debugger-Speicherfenster ist ein weiterer Zugriff und kann Cachewartung auslösen.
ETM betrachtet als Nächstes den Kontrollfluss.

Folie 17 / 42

Modul 3: Das ETM (Embedded Trace)

- Komprimierter Kontrollfluss, plus Sync, Ausnahme, manchmal Zeit
- Nicht „ein Bit pro Sprung“
- 400 MHz sind 400 Millionen **Takte**, nicht Instruktionen
- Hypothetisches Modell: 400e6 Ereignisse mal 4 Byte = 1,6 GB/s, nicht die echte ETM-Rate und nicht Gbit/s

Rekonstruktion braucht das passende Maschinenbild, die Protokollversion und einen vollständigen Abschnitt. Eine Lücke ist ein unbekannter Abschnitt, keine erfundene Gerade.

Notizen 17 / 42

ETM heißt Embedded Trace Macrocell. Sie liefert komprimierten Kontrollfluss, keine Kopie jeder Instruktion als langes Paket. ELF ist das Maschinenbild, mit dem der Decoder die Sprünge rekonstruiert. 400 MHz sind 400 Millionen Takte je Sekunde, nicht 400 Millionen Instruktionen. Die Rechnung 400 Millionen Ereignisse mal 4 Byte gleich 1,6 GB/s ist ein hypothetischer Überschlag, keine ETM-Bitrate. Instruction Trace enthält nicht automatisch einen Zeitstempel je Befehl. Cycle-accurate und Timestamp sind Extra-Fähigkeiten und werden nicht versprochen. Eine C-Zeile ist keine einzelne Instruktion. GCC und Optimierung verändern die Zuordnung.
Ein kleines Zweigbeispiel zeigt die Rekonstruktion.

Folie 18 / 42

ETM komprimiert den Kontrollfluss

Schema, kein echtes Paketformat. Der graue Abschnitt ist verloren und wird nicht interpoliert.



Eine C-Zeile kann mehreren Befehlen entsprechen. Ohne das ELF des **denselben** Builds rekonstruiert das Tool den Pfad nicht.
Sync-Verlust: Strecke grau lassen.

Notizen 18 / 42

Das Schema trennt C-Text, symbolische Thumb-Befehle, vereinfachte Traceereignisse und den Decoder mit der passenden ELF. Es ist kein echtes ETM-Paketformat. ELF ist das Image, GCC der Compiler. Die Option `-g` fügt Debuginfo hinzu und macht aus einer Zeile nicht genau einen Befehl. `-g` und `-O2` gehen zusammen. Gemessen wird das Image, das wirklich läuft, nicht ein zweiter Debug-Build. Inlining und wegoptimierte Variablen bleiben. Jeder Kern braucht seine eigene ELF. Ein grauer Abschnitt ist verlorene Synchronisation und wird nicht interpoliert. Nach nachgewiesener Resynchronisation kann ein späterer Abschnitt separat gelten.

Notizen, Fortsetzung 18 / 42

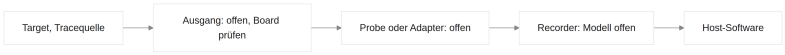
Die Ereignisse müssen über einen vorhandenen Ausgang zum Recorder.

Folie 19 / 42

Die TPIU und der physikalische Stecker

- TPIU ist ein möglicher Ausgang, nicht der Beweis, dass jede Spur dort ankommt
- Laboraufbau erst zeichnen, wenn Adapter, Active Probe und BlueBox-Modell feststehen
- iC5700: Ethernet-Angabe nicht als Gigabit übernehmen, solange die Gerätebeschreibung der **vorliegenden** Hardwareversion das nicht sagt
- FNet, Host-Ethernet und Target-Trace sind drei Kabelklassen
- Einschalten und Ausschalten nur nach der Anleitung des vorhandenen Geräts. Ohne Gerät ist das keine Laboranweisung.
- Keine Preisvergleiche

Schema der Rollen. Unbestätigte Kästen bleiben offen. Kein erfundener Stecker.



Notizen 19 / 42

TPIU, Trace Port Interface Unit, kann der Ausgang der Tracedaten auf dem Chip sein. Danach kommen Leitung, Probe oder Adapter, Recorder und Host. FNet und FBridge sind Produktwege des Zubehörs, kein beliebiges Host-Ethernet. Eine Geschwindigkeit der iC5700 übernehmen wir nicht, solange Hardwareversion und Anleitung fehlen. Das Bild hier zeigt Rollen. Ein verifizierter Aufbau nennt Geräte und Leitungen. Externe Digitaleingänge zeichnen wir nur, wenn das Modul vorhanden ist. Im Repository ist es nicht vorhanden. Ein- und Ausschaltreihenfolge kommt aus der Anleitung des realen Geräts, nicht aus dieser Folie. GND, VTref und Reset haben den Zweck, Pegel und Startzustand zu prüfen.
Die Zusammenfassung verbindet Quelle und zulässige Schlussfolgerung.

Folie 20 / 42

Zusammenfassung CoreSight

Quelle	erfasst	beweist nicht
ITM	geschriebene Stimuli	rückwirkungsfreie App
DWT	gefilterte CPU-Ereignisse	SRAM hinter dem Cache
ETM	Kontrollfluss, soweit komplett	jede C-Zeile in µs
DAP	momentane Sicht	Historie

Notizen 20 / 42

ITM-Softwarestimuli belegen, dass ein Marker ausgeführt und aufgenommen wurde. DWT zeigt konfigurierte Kernereignisse, nicht den ganzen SRAM und nicht DMA. ETM rekonstruiert Kontrollfluss nur in gültigen Abschnitten und mit dem passenden Maschinenbild. DAP liefert eine aktuelle Sicht auf einem benannten Zugriffsweg, keine Vergangenheit. CPU ist der Kern, dessen Zugriffe gemeint sind. Keine Quelle beweist allein, dass das System die Frist hält.
Frage: Warum beweist ein M7-Store keine M4-Sichtbarkeit? Weil eine Write-Back-Kopie und die MPU dazwischen liegen. In der nicht cachebaren Basis sieht der M4 den SRAM. In der absichtlich cachebaren Fehlervariante kann er den alten Wert lesen.
Es folgt, was die Zahlen für Anforderungen bedeuten.

Folie 21 / 42

Warum Systemanalyse und nachvollziehbare Nachweise wichtig sind

Normen motivieren Nachweise. Sie sind hier keine Rechtsauskunft.

- ISO 26262: Straßenfahrzeuge, nicht „jede Software ist illegal“
- DO-178C: Luftfahrt-Software, Einordnung über DO-330 und AC 20-115D
- IEC 61508: funktionale Sicherheit, nicht pauschal der Medizinstandard

Trace ist keines dieser Dokumente als einziges Pflichtwerkzeug. Aussagen nur mit Ausgabe und Abschnitt, wenn wir sie wirklich zitieren. Das tun wir hier nicht im Detail.

Notizen 21 / 42

Eine Anforderung braucht einen nachvollziehbaren Nachweis, nicht ein Fenster als Stempel. ISO, International Organization for Standardization, ordnet ISO 26262 der Straßenfahrzeug-Funktionsicherheit zu. IEC, International Electrotechnical Commission, ordnet IEC 61508 der funktionalen Sicherheit zu. FAA, Federal Aviation Administration, und AC, Advisory Circular, gehören zum Luftfahrtkontext. DO-178C ist der zugehörige Softwarestandard, DO-330 die Werkzeugqualifikation. Wir zitieren keine Klausel und behaupten keine Konformität. Die NASA-Quelle später ist ein historisches DO-178B-Tutorial. Der didaktische Punkt: Eine Messung beantwortet einen Ausschnitt und braucht benannte Bedingungen. Ein Tracefenster beweist nicht „ISO-konform“.
Die erste Timinggröße ist ein beobachtetes Maximum, keine WCET.

Folie 22 / 42

Metrik 1: Beobachtetes Laufzeitmaximum

$\max(C_{\text{beobachtet}})$ ist das Maximum **dieser** Läufe.
Es ist keine obere Schranke für alle zulässigen Ausführungen. Ein ungetesteter Pfad kann länger sein.
Synthetische Stichprobe, keine Messung: 12 µs, 15 µs, 14 µs. Das beobachtete Maximum ist 15 µs und keine WCET.
Der RM-Test bleibt der aus VL2: hinreichend $n(2^{1/n}-1)$, Grenzwert $\ln 2 \approx 69{,}3\,\%$ für große n. Ein Messmaximum ohne konservative Absicherung ist kein C_i für diesen Beweis.

Notizen 22 / 42

WCET, Worst-Case Execution Time, ist die obere Grenze über alle zulässigen Ausführungen unter festgelegten Annahmen. Das Maximum einer endlichen Stichprobe ist eine andere Zahl. Pfad, Eingabe, Cache, Interruptlast und die Messdefinition ändern sie. Eine kleine synthetische Stichprobe zeigt einen größten Balken und keine Garantie für die unbekannte Menge. RM, Rate Monotonic, ist das Scheduling nach der Periode und nicht die Handbuchnummer RM0399. Der hinreichende Test aus Vorlesung 2 lautet $n(2^{1/n}-1)$ und nähert sich für großes n dem Wert $\ln 2$, etwa 69,3 Prozent. Er ist hinreichend, nicht notwendig. Ein Messmaximum ohne konservative Begründung ist kein C_i für diesen Beweis. Eine Million Läufe zeichnet kein garantiertes Ende. Qualität heißt: Sync, passende ELF, keine Lücken in der Summe, bekannte Zeitbasis.
Fristen beziehen sich auf die Antwort ab Freigabe, nicht nur auf einen Funktionsabschnitt.

Folie 23 / 42

Metrik 2: Deadline-Verifizierung

Job: Freigabe r, Start s, Fertig f, Deadline $d = r + D$.

$$R = f - r, \qquad \text{Slack} = d - f$$

Ein negativer Slack verletzt **diese** Frist. Daraus folgt kein „illegal im Straßenverkehr“.
OS-Awareness kennt Freigabe und Deadline nicht von selbst. Die Ereignisse müssen im Trace oder im Protokoll liegen.

Notizen 23 / 42

Ein Job ist eine Aktivierung einer Task. r ist die Freigabe, s der Beginn der eigenen Arbeit, f die Fertigstellung, D die relative Deadline, $d = r + D$ die absolute, $R = f - r$ die Antwortzeit, Slack gleich $d - f$. OS ist das Betriebssystem. Ein negativer Slack verletzt diese Frist und ist kein Straßenverkehrsurteil. Das Tool kennt d nur, wenn wir sie konfigurieren oder aus Ereignissen rekonstruieren. Eine laufende Taskspur enthält Freigabe und Deadline nicht von selbst. Das gehaltene Beispiel der nächsten Folie hat

Notizen, Fortsetzung 23 / 42

Slack 1 ms. Eine Variante mit Fertigstellung nach d hätte negativen Slack und muss als solche beschriftet sein.
Bei einer externen Reaktion gibt es mehrere Latenzabschnitte.

Folie 24 / 42

Metrik 3: Interrupt-Latenz

Kette, nicht eine Zahl:

1. Flanke → ISR-Eintritt
2. ISR-Laufzeit
3. Veröffentlichung → Task Ready
4. Ready → Task-Start
5. Start → sichtbare Reaktion

Folie zur „ersten Bottom-Half-Zeile“ meint eine **Dispatchlatenz** bis zu einem definierten Maschinenpunkt, nicht die IRQ-Eintrittslatenz und nicht die End-to-End-Reaktion.
Synthetisches Beispiel, keine Adaptermessung. Uhr hypothetisch gemeinsam.

Marke	Zeit
externe Flanke	100 µs
ISR-Eintritt	106 µs
Veröffentlichung	110 µs
Task Ready	111 µs
ISR-Ende	115 µs
Task beginnt	120 µs
physische Reaktion	128 µs

Daraus: 6 µs Eintritt, 9 µs ISR-Intervall, 9 µs Ready bis Start, 28 µs Ende zu Ende. Die Summe der disjunkten Stücke 6 + 4 + 1 + 9 + 8 ist 28. Die vollen 9 µs ISR nicht noch einmal addieren. Eine Messung ist keine obere Schranke. „Erste C-Zeile“ ist bei `-O2` unscharf.

Notizen 24 / 42

IRQ ist die Anforderung, ISR die Behandlungsroutine. Bottom Half ist die aufgeschobene Arbeit, hier oft eine M4-Task. Die Kette hat mehrere Endpunkte: Flanke bis ISR-Eintritt, ISR-Dauer, Veröffentlichung bis Ready, Ready bis erster Maschinenpunkt der Task, Start bis physische Reaktion. Diese Abschnitte überlappen. Eine Veröffentlichung kann innerhalb der ISR liegen. Man addiert nicht die volle ISR-Zeit noch einmal zu allen Teilintervallen. Die gemeinsame Uhr oder die Korrelation von externer Flanke und internem Stempel muss benannt sein. „Erste C-Zeile“ ist bei `-O2` kein eindeutiger Maschinenpunkt. Welches Intervall das Zubehör sieht, bleibt offen, bis der Adapter feststeht. Eine Messung ist keine obere Schranke. Das Jobbeispiel trennt eigene CPU-Arbeit von der verstrichenen Antwortzeit.

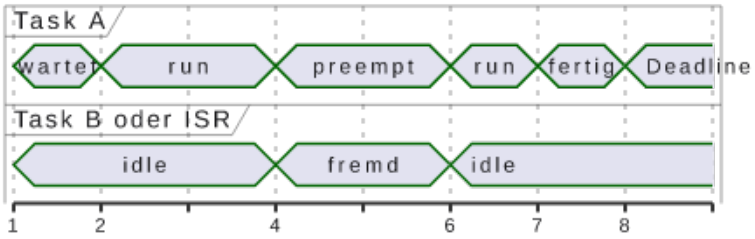
Folie 25 / 42

Netto-Rechenzeit vs. Brutto-Antwortzeit

Lehrbeispiel, Overhead in diesem Bild **0**:

Marke	Zeit
r	1 ms
Running	2–4 ms
Präemption	4–6 ms
Running	6–7 ms
f	7 ms
d	8 ms

CPU-Zeit des Jobs: 3 ms. $R = 6$ ms. Slack: 1 ms.
Lehrdiagramm, Overhead 0, keine Messung. Eine Spur je Teilnehmer.



Zeit	Task A	fremde Arbeit
1–2 ms	wartet nach Freigabe	—
2–4 ms	läuft, 2 ms	—
4–6 ms	verdrängt	2 ms
6–7 ms	läuft, 1 ms	—
7 ms	fertig	Deadline bei 8 ms

Das ist nicht die WCET.

Notizen 25 / 42

Die Tabelle ist ein synthetisches Modell mit Overhead null. CPU ist die eigene Rechenzeit von Task A: $(4-2)+(7-6) = 3$ ms. Die Antwortzeit ist $f - r = 6$ ms, der Slack $d - f = 1$ ms. Anfangswarten 1 ms plus Präemption 2 ms plus eigene Arbeit 3 ms ergeben dieselben 6 ms. ISR-Zeit wird A nicht zugeschlagen, wenn sie auf der Systemspur liegt. Nach f kann der Thread weiterleben. Die Jobdauer ist nicht die Tasklebensdauer. Die 3 ms sind die Zeit dieses Modelljobs, keine WCET.
Reale Kontextwechsel brauchen Arbeit, die dieses Bild bewusst auf null setzt.

Folie 26 / 42

Metrik 4: Context Switch Overhead

PendSV ist ein Teil. Dazu können Exception-Eintritt, Rückkehr, FPU, Scheduler und die Präemption selbst gehören.
Dieselbe Zeit nicht zweimal in die Antwortzeitanalyse zählen.
2,5 µs ist ein Beispiel, keine Kennzahl des H755.

Notizen 26 / 42

PendSV ist die aufschiebbare Exception, über die der FreeRTOS-Port den Kontextwechsel erledigt. Der Handlerumfang ist nicht automatisch das Intervall zwischen dem letzten Befehl von Task A und dem ersten von Task B. Register, Stack, Scheduler, Exception-Eintritt, FPU-Kontext und verschachtelte Ereignisse können dazugehören, je nach Vertrag. FPU ist die Gleitkommaeinheit. Tail-Chaining und Lazy-FPU sind optionale Vertiefungen, keine feste Dauer. Die 2,5 µs sind synthetisch und kein H755-Benchmark. Overhead entweder separat modellieren oder in den gemessenen Größen belassen, nicht doppelt addieren. Anfang und Ende müssen im Trace benannte Kanten sein.
Traces können außerdem Pfade für die Testabdeckung sichtbar machen.

Folie 27 / 42

Ausblick: Code Coverage (Testabdeckung)

`if (a && b)`, Kurzschluss: bei `a == false` wird `b` nicht ausgewertet.

Test	Eingabe a	Eingabe b	ausgewertetes a	ausgewertetes b	Entscheidung
T1 TT	true	true	true	true	wahr
T2 TF	true	false	true	false	falsch
T3 FT	false	true	false	nicht ausgewertet	falsch

Bei `a == false` wird `b` nicht ausgewertet. Die Eingabe `b = true` im FT-Test ist kein beobachtetes Laufzeitergebnis. Unabhängigkeit im Booleschen Modell: für `a` das Paar TT/FT, für `b` das Paar TT/TF.
Statement Coverage zählt ausführbare Anweisungen, nicht Textzeilen. Decision ist nicht Condition. MC/DC braucht diese Paare und definierte Regeln bei Kurzschluss.
Instruction Trace plus Mapping ist kein automatischer MC/DC-Nachweis. Optimierung verändert das Bild. Coverage zeigt nicht, dass die Funktion richtig ist oder die Frist hält.
Die NASA-Notiz ist ein Tutorial im Kontext von DO-178B. Das ist keine „absolute Pflicht“ für dieses Labor.

Notizen 27 / 42

MC/DC, Modified Condition/Decision Coverage, verlangt den unabhängigen Einfluss jeder Bedingung nach einer benannten Regel. NASA, National Aeronautics and Space Administration, beschreibt das im Tutorial NASA/TM-2001-210876, Abschnitt 3.6, im historischen DO-178B-Kontext. Daraus folgt keine Laborpflicht und kein Werkzeugstempel. Statement Coverage zählt ausführbare Anweisungen. Decision Coverage die beiden Ausgänge. Condition Coverage true und false der ausgewerteten Bedingungen. Im Beispiel `if (a && b)` wertet Test FT die Bedingung b nicht aus. Die Eingabe b gleich true ist der Testvertrag, kein Traceereignis. Die Paare TT/FT und TT/TF zeigen den Einfluss von a und b im Booleschen Modell. ETM-Kontrollfluss mit Quellzuordnung ist allein kein MC/DC-Nachweis.
Das Werkzeug muss Trace aufnehmen und dem Build zuordnen.

Folie 28 / 42

Das Werkzeug: In-Circuit Emulator (ICE)

- BlueBox und winIDEA nur in der Version, die im Labor steht
- Modell, Active Probe, Adapter und Zusatzmodule aufschreiben, bevor eine Folie sie zeichnet
- Hier: Liste offen

Notizen 28 / 42

ICE, In-Circuit Emulator, bezeichnet historisch ein Gerät, das den Prozessor ersetzte. Ein heutiger Debugger oder Recorder muss das nicht. Er braucht Debugadapter, gegebenenfalls eine Active Probe, Aufnahme, Puffer und Analysesoftware. Die Liste offener Geräte ist eine Konfigurationstabelle, kein Beweis, dass ein Modell im Labor steht. Herstellerseiten widersprechen sich bei Schnittstellen. Die Hardwareversion und die Anleitung gewinnen. Eine Verbindung und eine unterstützte Tracequelle sind zwei Prüfungen.
BlueBox ist eine Produktfamilie für solche Funktionen, kein belegtes Kursgerät.

Folie 29 / 42

Vorstellung: TASKING BlueBox

Offen, bis das Gerät da ist:

- genaue Typbezeichnung
- ob paralleler Trace, SWO oder nur Debug
- Ethernet der Box, nicht raten
- keine Gigabit-Annahme
- keine externen ADIO-Pins ohne Modul

Notizen 29 / 42

Die BlueBox steht im Schema als Recorderrolle. Ein bestimmtes Modell ist nicht belegt. SWO ist der

Notizen, Fortsetzung 29 / 42

serielle Traceausgang, nicht der SWD-Datenkanal. ADIO, falls das Handbuch es für das vorhandene Modul so nennt, wäre ein digitaler Erfassungsweg. Ohne Modul erfinden wir keine Portfunktion. Debugzugriff, Instruction- und Data-Trace, externe Erfassung und Hostübertragung sind getrennte Fähigkeiten. FBridge und FNet sind Anschlüsse zum Zubehör, kein austauschbares Host-Ethernet, auch wenn Stecker ähnlich aussehen. Die Konfigurationstabelle hat Spalten für Gerät, Revision, Probe, Adapter, Tracepfad und Zeitquelle. „Offen“ nennt die Prüfstelle. Der didaktische Ersatz ist der lokale Zähler oder GPIO.
winIDEA verbindet Recorderdaten, Maschinenbild und Auswertung, sobald eine Installation feststeht.

Folie 30 / 42

Die IDE: winIDEA

Menüpfade der **installierten** Version. „Plugins → FreeRTOS“ nicht als ewig gültig abschreiben. Geladen wird die ELF des gemessenen Builds, inklusive DWARF, für jeden Kern die eigene Datei. Hash oder Build-ID notieren.

Notizen 30 / 42

IDE, Integrated Development Environment, ist hier winIDEA als Analyseumgebung. ELF ist das Image, DWARF die Debuginformation darin. Arbeitsbereich, Ziel, Symboldatei, Recorder und Analysedarstellung sind Funktionen. Menüpfade gelten nur für eine bestätigte Version. Die online geprüfte Hilfe 9.26.20 ist nicht die Kursinstallation. Je Kern wird die ELF geladen, die zum laufenden Image passt. Hash oder Build-ID identifizieren den Build. Zwei ELF-Container können verschiedene Hashes haben, obwohl das Maschinenbild gleich ist, weil Debugmetadaten dazukommen. Ein neu gebauter Debuglauf kann andere Zeiten und Adressen haben als der optimierte Lauf. Entscheidend sind Buildzuordnung und die wirklich geladenen Bereiche.
Die nächste Folie trennt die Inhalte der ELF.

Folie 31 / 42

ELF: Maschinenbild, Symbole und Debugzuordnung

- Adressen, Typen, manchmal Zeilen
- mehrere Befehle je Zeile, Zeilen ohne Code
- der Decoder braucht dasselbe Image wie der Chip
- ein anderes ELF erklärt den Trace falsch, ohne dass das Tool „lügt“

Notizen 31 / 42

ELF trägt das Maschinenbild, Symbole und optional DWARF. DWARF, Debugging With Attributed Record Formats, liefert Typen und Zeilen. Der Kontrollflussdecoder braucht das Maschinenbild und das Protokoll. Symbole und DWARF machen daraus Funktions- und Quellnamen. Nicht jedes ELF enthält vollständige Debugdaten. Optimierung verschiebt die Zuordnung. Eine falsche Builddatei erzeugt plausible, aber

Notizen, Fortsetzung 31 / 42

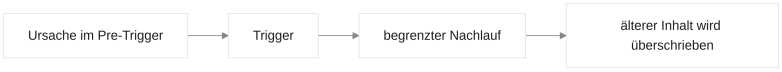
falsche Namen. Fehlende Sync-Pakete machen den Abschnitt für Summen und Coverage ungültig. Das ist eine Eigenschaft der Aufnahme, nicht der ELF. Nach neuer Synchronisation kann ein späterer Abschnitt separat gelten. Die Lücke selbst bleibt unbekannt.
Der Profiler bestimmt danach, welchen Ausschnitt er aufzeichnet.

Folie 32 / 42

Der Profiler und Trigger

- Trigger brauchen freie Komparatoren und eine Route
- `error_flag == 1` ist ein Beispiel, keine eingebaute Fähigkeit jeder Box
- „10 Sekunden alles“ hängt an Puffer und Rate
- Für die Ursache oft ein Ring mit Pre-Trigger, nicht erst ab dem Fehler starten

Synthetisches Beispiel, keine Gerätekapazität: 2 MiB bei 400 KiB/s ergeben 5,12 s vor Overhead.



Notizen 32 / 42

Ein Trigger ist die Bedingung, die Aufnahme startet, stoppt oder nur markiert. Pre-Trigger ist die gespeicherte Historie davor, typisch in einem Ringpuffer. DWT kann eine Datenbedingung liefern, ETM den Kontrollfluss. `error_flag == 1` ist ein Beispiel. Die Umsetzung braucht Adresse, Breite und einen freien Komparator. Ein Wertvergleich kann eine weitere Ressource kosten. Die Rechnung 2 MiB bei 400 KiB/s gleich 5,12 s ist synthetisch und keine BlueBox-Kapazität. Vor der Auswertung prüfen wir Sync, passende ELF, Overflow-Hinweise und die Zeitbasis. Ein fehlender Overflow-Hinweis beweist ohne passende Quelle keine Vollständigkeit.
Rohereignisse brauchen danach den Betriebssystemkontext.

Folie 33 / 42

Das Problem der OS-Sichtbarkeit

`pxCurrentTCB` im lokalen M4-Kernel: die **ausgewählte** Task.
Während der ISR zeigt der Zeiger oft weiter auf sie, obwohl kein Taskcode läuft.
Der Wechsel liegt im Context Switch und ist nicht zwingend der erste Zyklus der neuen Task.
Ein wiederverwendeter TCB-Pointer ist keine ewige ID. Layout der eingesetzten Kernelversion prüfen. SMP-Strukturen sind hier nicht der M4-Kernel.

Notizen 33 / 42

OS ist das Betriebssystem, TCB der Task Control Block. `pxCurrentTCB` gehört zum Single-Core-M4-Kernel und zeigt auf den ausgewählten Taskblock. Während ISR oder PendSV kann der Zeiger weiter

Notizen, Fortsetzung 33 / 42

auf eine Task zeigen, obwohl der Kern deren Code nicht ausführt. Der Wechsel des Zeigers ist nicht automatisch der erste Taskbefehl. Der Port und der Decoder müssen die Grenze liefern. Wiederverwendete Adressen brauchen eine Lebensdauer. SMP-Strukturen erklären diesen M4-Kern nicht. Ein Running-Profil ist die Ausführungshistorie, nicht die Ready- und Blocked-Historie.
OS-Awareness kombiniert mehrere Informationen und hat Ausbaustufen.

Folie 34 / 42

OS-Awareness (Betriebssystem-Bewusstsein)

Zwei Stufen:

1. Running: aus `pxCurrentTCB` und dem Trace, ISR und PendSV daneben
2. Zustände Ready/Blocked: zusätzliche Instrumentierung, Aufwand kennzeichnen

Debug-Listen sind keine lückenlose Live-Historie.
ORTI/OSEK bleibt etwas anderes als FreeRTOS-Awareness. Queues können extra Registrierung brauchen.

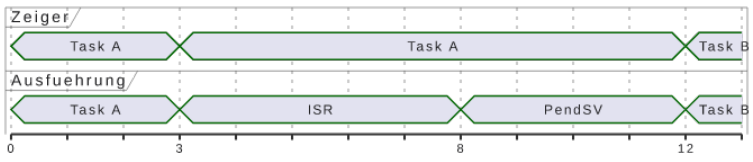
Notizen 34 / 42

Drei Darstellungen bleiben getrennt. Die Debuggerliste zeigt aktuelle Kernelobjekte. Das Running-Profil zeigt, welche Task oder ISR in gültigen Ereignissen lief. Ein Zustandstrace mit Ready und Blocked braucht zusätzliche Instrumentierung. Ready heißt ausführungsbereit, Blocked heißt Warten, Running heißt tatsächliche Ausführung unter der gewählten Zuordnung. Der Taskzeiger allein liefert nicht alle drei. Während einer ISR zeigt er oft die zuletzt ausgewählte Task, nicht „die ISR ist diese Task“. ORTI, OSEK Run Time Interface, ist ein anderer Beschreibungsstandard und kein FreeRTOS-Plugin. Kernelversion und registrierte Queues werden geprüft.
Ein Lehrschema zeigt das, ohne einen Laborscreenshot vorzutäuschen.

Folie 35 / 42

Wie winIDEA FreeRTOS visualisiert

Lehrschema, keine Laboraufnahme. Menüs gelten nur für die installierte Hilfe.



Beide Kernspuren gleichzeitig nur, wenn beide Quellen wirklich korreliert aufgenommen werden. Sonst zwei Aufnahmen und keine Subtraktion der Zeitachsen.

Notizen 35 / 42

Das Bild ist ein Lehrschema, keine Laboraufnahme. Task A, Task B, ISR und PendSV haben eigene Spuren. Ready und Blocked erscheinen nur, wenn sie instrumentiert sind. Ein Running-Profil wird nicht stillschweigend zur Zustandshistorie. RTOS ist das Echtzeitbetriebssystem auf dem M4. Zwei Kernspuren auf einer Achse brauchen gleichzeitige Aufnahme und Korrelation. Sonst bleiben getrennte Achsen. Unabhängige Zykluszähler werden nicht voneinander abgezogen. Fehlende Screenshots ersetzen wir durch dieses Schema, nicht durch ein nachgemaltes Fenster.
Ein anderer Standard beschreibt OS-Objekte im Automotive-Kontext.

Folie 36 / 42

Der OSEK/ORTI-Standard (Automotive)

- ORTI beschreibt OSEK/AUTOSAR-OS-Objekte
- FreeRTOS-Awareness ist ein anderes Plugin und ein anderes Layout
- Beides nicht vermischen, auch wenn beide „Tasks“ zeichnen

Notizen 36 / 42

OSEK steht für offene Systeme und deren Schnittstellen in der Automobilelektronik. ORTI beschreibt Lage und Bedeutung von Betriebssystemobjekten für Werkzeuge. Diese Metadaten erzeugen keine Ereignishistorie. AUTOSAR, AUTomotive Open System ARchitecture, nutzt eigene OS-Schnittstellen. Eine ähnliche Taskliste ist nicht dieselbe Schnittstelle. Einen neueren Standard wie ARTI nennen wir nur als Ausblick, ohne ihn als Laborpflicht zu setzen. Für dieses Labor zählt die lokale M4-FreeRTOS-Instanz, nicht ein AUTOSAR-OS.
Die Konzepte werden nun in Messversuchen angewendet.

Folie 37 / 42

Laborvorbereitung: Tracing im Einsatz

Ohne Spezialhardware: GPIO um die Funktion oder `CYCCNT` desselben Kerns. Lehrumrechnung, keine Boardmessung: bei 400 MHz sind 1000 Takte 2,5 µs, bei 200 MHz 5 µs. Den Zähler des anderen Kerns nicht subtrahieren.

```
/* Ungebautes Lehrbeispiel. CYCCNT nur, wenn der Kern ihn hat. *
uint32_t before = DWT->CYCCNT;
uint32_t result = work_under_test();
uint32_t after = DWT->CYCCNT;
uint32_t delta = after - before; /* kurz, kürzer als ein Umlauf
(void)result; /* Ergebnis benutzen, damit der Compiler work nich
```

`work_under_test` ist eine benannte Funktion mit verwendetem Ergebnis, keine leere Hülle. Die Differenz enthält Unterbrechungen, solange der Zähler läuft. Reine Task-CPU-Zeit braucht eine Running-Zuordnung. Kein `printf` zwischen den beiden Lesungen.
Versuch A: eine `uint32_t`-Variable mit alleinigem Schreiber, Werte 0, 1, 2 nach dokumentierter Init. Ein fehlendes Anfangspaket ist nicht der Wert 0.
Versuch B: non-cacheable Ping-Pong aus VL6. Druck 1, 3, 5. Die 500 ms liegen außerhalb der IPC-Messung.
Versuch C: separates Image, D-Cache an, Region nachweislich Write-Back. Hypothese, kein Drehbuch: M7-Store 1 kann da sein, M4 liest weiter 0. Kein garantierter Freeze. Timeout und Abbruch wie in VL6.
Versuch D: ein Job mit definierten Kanten. Min, Max, Anzahl, Auflösung, Bedingungen. Verluste machen die Summe ungültig.

Notizen 37 / 42

Der Basisteil braucht keinen Recorder. CYCCNT ist der lokale DWT-Zykluszähler des gewählten Kerns, keine zweite Systemuhr. Zwei `uint32_t`-Stände, `after - before`, gelten im Modell, solange das Intervall kürzer als eine Zählerperiode ist. Bei den Lehrfrequenzen 400 MHz und 200 MHz sind 1000 Takte 2,5 µs beziehungsweise 5 µs. Die 32-Bit-Periode liegt dann bei etwa 10,74 s beziehungsweise 21,47 s. Das sind keine bestätigten Boardtakte. Der Zähler umfasst Interrupts, wenn er weiterläuft. `printf` steht nicht zwischen den Lesungen. Das Ergebnis wird lokal gehalten. Der Code ist ungebaut, bis die CMSIS-Header des Ziels die Register bestätigen.
Versuch A vergleicht einen bekannten Anfang mit einem fehlenden ersten Paket. Ein Default 0 ist keine Beobachtung. Versuch B nutzt den nicht cachebaren Ping-Pong aus Vorlesung 6. Der M7-Roundtrip misst mit einer Uhr um Veröffentlichung und Antwort. Die 500-ms-Pause liegt außerhalb. Getrennte M7- und M4-Aufnahmen ergeben ohne Korrelation keine Inter-Core-Latenz. Versuch C ist ein separater Write-Back-Build, kein korrektes IPC. Versuch D berechnet CPU-Zeit, Antwortzeit und Slack nur über vollständige Jobkanten.
Das M4-Profil wird mit klarer Informationsgrenze eingerichtet.

Lehrversuche A bis D, zu Folie Labor

Jeder Kasten ist ein Vertrag. Erwartete Werte sind Lehrannahmen, keine Targetmessung. Abbruch: Timeout wie in Vorlesung 6, Shared-Block nicht neu initialisieren, solange die Gegenstelle läuft.

Feld	A lokale Variable	B Ping-Pong	C Cachefehler	D ein Job
Frage	fehlt der Anfang?	Roundtrip einer Uhr	sieht M4 den Store?	CPU, Antwort, Slack
Kern	ein Kern, alleiniger Schreiber	M7 misst, M4 antwortet	separates M7-Image	M4-Task
Anfang/Ende	vor erstem Schreiben und nach 0, 1, 2	um Veröffentlichung und Antwort	Store und benannter M4-Leseweg	Freigabe bis Jobende, Running-Abschnitte
Quelle	DWT oder synthetische Tabelle	Marker oder CYCCNT des M7	DWT-Store und M4-Beobachtung	Kernelereignis oder CYCCNT/GPIO
Uhr	lokale Zählung, Frequenz prüfen	dieselbe M7-Uhr	nicht zwei CYCCNT subtrahieren	eine benannte Uhr
Enthalten	nur diese Schreibfolge	M4-Arbeit, Poll, Rückgabe	kein Logging im Fenster	Interrupts nur, wenn die Definition sie einschließt
Abbruch	kein Paket als 0 lesen	500 ms Pause außerhalb	Cachewartung aus	Lücken aus der Summe

Versuch A ohne DWT: eine synthetische Ereignistabelle zeigt „kein Paket“ und „Paket 0“ als zwei Zeilen. Versuch B behält `ipc.magic`, `ipc.state`, `ipc.payload` und die nicht cachebare MPU-Basis. Druck 1, 3, 5 ist das Protokoll. Versuch C prüft CCR, Shareability und Überlappung. Weiterhin shareable kann die Cachewirkung verhindern. Versuch D nennt Min, beobachtetes Max, Anzahl, Einheit, Auflösung und die Zahl ausgeschlossener Jobs. Das Maximum ist keine WCET.

Notizen 38 / 42

Die vier Verträge gehören zur Laborfolie. A trennt einen aufgenommenen Wert 0 von einem fehlenden Anfang. B ist der nicht cachebare Kanal aus Vorlesung 6. Der M7-Roundtrip enthält Bearbeitung, Polling und Rückgabe und ist keine isolierte Einweglatenz. C ist ein Fehlerexperiment in einem eigenen Build. Ein ausbleibender Fehler beweist die Korrektheit nicht. D rechnet eigene CPU-Zeit, Antwortzeit und Slack nur über vollständige Kanten. Ein CYCCNT- oder GPIO-Intervall um eine präemptierbare Funktion heißt

Notizen, Fortsetzung 38 / 42

Intervall, nicht reine Taskzeit.
Danach wird das M4-Running-Profil mit eigener Spur für ISR und PendSV eingerichtet.

Folie 39 / 42

Schritt 3–4: FreeRTOS Awareness & Profiler

Running-Profil der M4-Task: wiederkehrendes Delay ist Blocked/Ready/Running im Polling, kein eingefrorener Kern.
`osDelay(1)` ist ein Tick. Die Poll-Latenz ist nicht nur die Umschaltzeit.
Zustandshistorie nur mit der zusätzlichen Konfiguration und als instrumentierte Variante beschriftet.

Notizen 39 / 42

Der Ablauf ist: Build zuordnen, Beobachtungspfad bestätigen, Trace konfigurieren, Running-Profil einrichten, Zustandstrace nur optional. RTOS ist FreeRTOS auf dem M4. `osDelay(1)` blockiert für mindestens einen Kernel-Tick. Die Dauer des Ticks steht im Ziel, nicht in dieser Folie. Die Task wird blockiert und später bereit. Ein Running-Profil belegt nicht jede interne Zustandskante. ISR und PendSV bekommen eigene Spuren. Ihre Zeit wird der Task nicht als eigene CPU-Zeit zugeschlagen. Eine Pollreaktion enthält Pollintervall, Ready-Wartezeit und Bearbeitung, nicht nur den Kontextwechsel. TCB ist der Taskblock, dessen Zeiger während der ISR stehen bleiben kann.
Erst nach der Qualitätsprüfung beginnt die Auswertung.

Folie 40 / 42

Schritt 5: Die Aufnahme & Analyse

Diagnose, je eine Prüfung:

Symptom	Prüfung
keine Verbindung	Versorgung, Reset-Reihenfolge, richtiger Adapter
nur ein Kern	ELF und Core-Zuordnung
keine Datenpakete	Komparator, Adresse, Breite, Startzeit
falsche Achse	Zeitquelle, nicht nur 400 MHz
Anfang fehlt	nicht als 0 werten
Verluste	Lücke markieren; Summen nur über vollständige Abschnitte. Nach neuer Sync kann ein späterer Abschnitt separat gelten
falsche Tasks	Kernelversion, TCB-Layout
nur Running	State-Profiling fehlt
Cachefehler bleibt aus	CCR, MPU, Neustart, Debug-Cachewartung aus

Notizen 40 / 42

Die Tabelle ist eine Folge von Hypothesen, keine feste Zuordnung Symptom gleich Ursache. Zuerst Aufbau, Kern, Build und Zeitquelle, dann Filter und Datenqualität, zuletzt die abgeleiteten Größen. ELF muss zum laufenden Image passen. Ein fehlender Anfangswert bleibt unbekannt und wird nicht als 0 gelesen. Verlustintervalle werden markiert. Summen laufen nur über vollständige Abschnitte. Nach neuer Synchronisation und bekannter Kontextzuordnung kann ein späterer Abschnitt separat gelten. CCR und MPU entscheiden im Cacheversuch, ob die Region wirklich Write-Back ist. Live-Memory und automatische Cachewartung können Versuch C verfälschen. Sie bleiben aus oder werden als Störung protokolliert.
TCB-Wechsel während einer ISR ist keine Taskausführung.
Die Schlussfolie fasst Messdefinition und zulässige Aussage zusammen.

Folie 41 / 42

Zusammenfassung des Tages

- Messfrage, Quelle, Grenze
- 434 µs Leitungszeit, 3 ms CPU, 6 ms Antwort, 1 ms Slack im Lehrjob
- DWT am CPU-Zugriff, nicht als SRAM-Monitor
- ETM rekonstruiert Pfade, solange das Bild vollständig ist
- $\max(C)$ ist kein WCET-Beweis
- Hardwareliste und gleichzeitige Dual-Core-Traces bleiben offen

Notizen 41 / 42

Eine zulässige Aussage nennt Messfrage, Quelle, Zeitbasis, Datenqualität und Grenze. Die 434 µs sind die UART-Leitungsdauer von fünf Bytes bei 8N1 und 115200 bit/s, nicht die CPU-Zeit von `printf`. Im Lehrjob sind 3 ms eigene CPU-Zeit, 6 ms Antwortzeit und 1 ms Slack. Ein DWT-Schreibereignis ist kein SRAM-Inhalt und keine M4-Sicht. ETM-Kontrollfluss gilt nur in synchronen Abschnitten. Ein Stichprobenmaximum ist keine WCET. Zwei CYCCNT-Werte verschiedener Kerne werden nicht subtrahiert. Frage: Warum ist das größte Messergebnis kein Beweis? Weil andere Pfade länger sein können. Frage: Wann darf man nach einem Verlust wieder rechnen? Wenn Sync, Kontext und Zeitbasis für einen neuen Abschnitt stehen. Die Lücke bleibt draußen.
Der nächste Termin behandelt die Hardware-Semaphore zwischen den Kernen. Die Quellenliste und die offene Gerätekonfiguration stehen im Anhang, nicht als Folienbehauptung. Geprüft am 7. Oktober 2026: Cortex-M7 TRM, CMSIS-ITM-Beschreibung, UM2408 Rev. 6, RM0399 und H755-Dokumentation, winIDEA-Hilfe 9.26.20 als Online-Stand, NASA/TM-2001-210876, FAA AC 20-115D. Die Kursinstallation und das Tracezubehör bleiben offen.

Anhang: offene Konfiguration, zu Folie 40

Keine Laborbehauptung. Spalten: Lehrannahme, im Kursaufbau zu prüfen, Prüfstelle.

Punkt	Lehrannahme	zu prüfen	Prüfstelle
Bauteil	STM32H755, Nucleo aus VL6	Boardrevision	Aufdruck, UM2408
Firmware	M7 Bare Metal, M4 FreeRTOS	ELF je Kern, Optimierung	Build, Map
Takt	400 und 200 MHz nur als Rechnung	Clock-Tree	Cube-Clock, Register
CYCCNT	Zähler des einen Kerns	Freigabe, Halt, Sleep	TRM, CMSIS-Header
Tracepfad	Konzept ITM, DWT, ETM	Pin, Adapter, Recorder	Schaltplan, Gerät
BlueBox, winIDEA	Rollen, keine Modelle	Version 9.26.20 ist nur online	Installation
Zeitkorrelation	eine Uhr je Messung	zwei Kerne gemeinsam	Aufnahmeprotokoll

Selbstkontrolle, kurz: 434 µs sind Leitungszeit. Ein Breakpoint kann einen Race verschieben. Viele IRQ-Flanken im Halt erzeugen nicht viele Stackframes. `ITM_SendChar` kann warten. Ein DWT-Store 1 ist kein M4-SRAM. Ein fehlendes Paket ist nicht 0. Der ETM-Decoder braucht dasselbe Maschinenbild. Zwei CYCCNT verschiedener Kerne sind nicht eine Differenz. Der Lehrjob hat 3 ms eigene CPU-Zeit und 6 ms Antwortzeit. Decision Coverage ersetzt Condition Coverage nicht, weil im FT-Test b nicht ausgewertet wird. Nach einem Verlust gilt erst der neu synchronisierte Abschnitt.

Notizen 42 / 42

Dieser Anhang gehört zur Schlussfolie und ist kein zusätzlicher Stoff. Die Tabelle hält fest, was die Vorlesung annimmt und was am Gerät noch fehlt. Eine Lehrannahme ist keine Messung der Kategorie C. Die Selbstkontrolle wiederholt die Grenzen: Leitung gegen CPU-Zeit, Halt gegen Pending, Stimulus gegen den ganzen ITM-Baustein, Store gegen Sichtbarkeit, Stichprobe gegen WCET, Zeiger gegen Ausführung, Kurzschluss gegen beobachtetes b.
Damit endet der Termin. Die Hardware-Semaphore folgt als eigener Synchronisationsbaustein.